

PORTAL DE CLIENTE · DOCUMENTACIÓN TÉCNICA

EL PORTAL POST-PURCHASE DE LIT.

Cómo está construido, qué hace cada superficie, y las decisiones técnicas detrás del producto.

SUPERFICIES

08

Hub, Cuenta, Skip, Plan,
Cancel, Login, Address,
Pago

ENDPOINTS API

20+

Auth, subscription, hub,
drops, webhooks

INTEGRACIONES

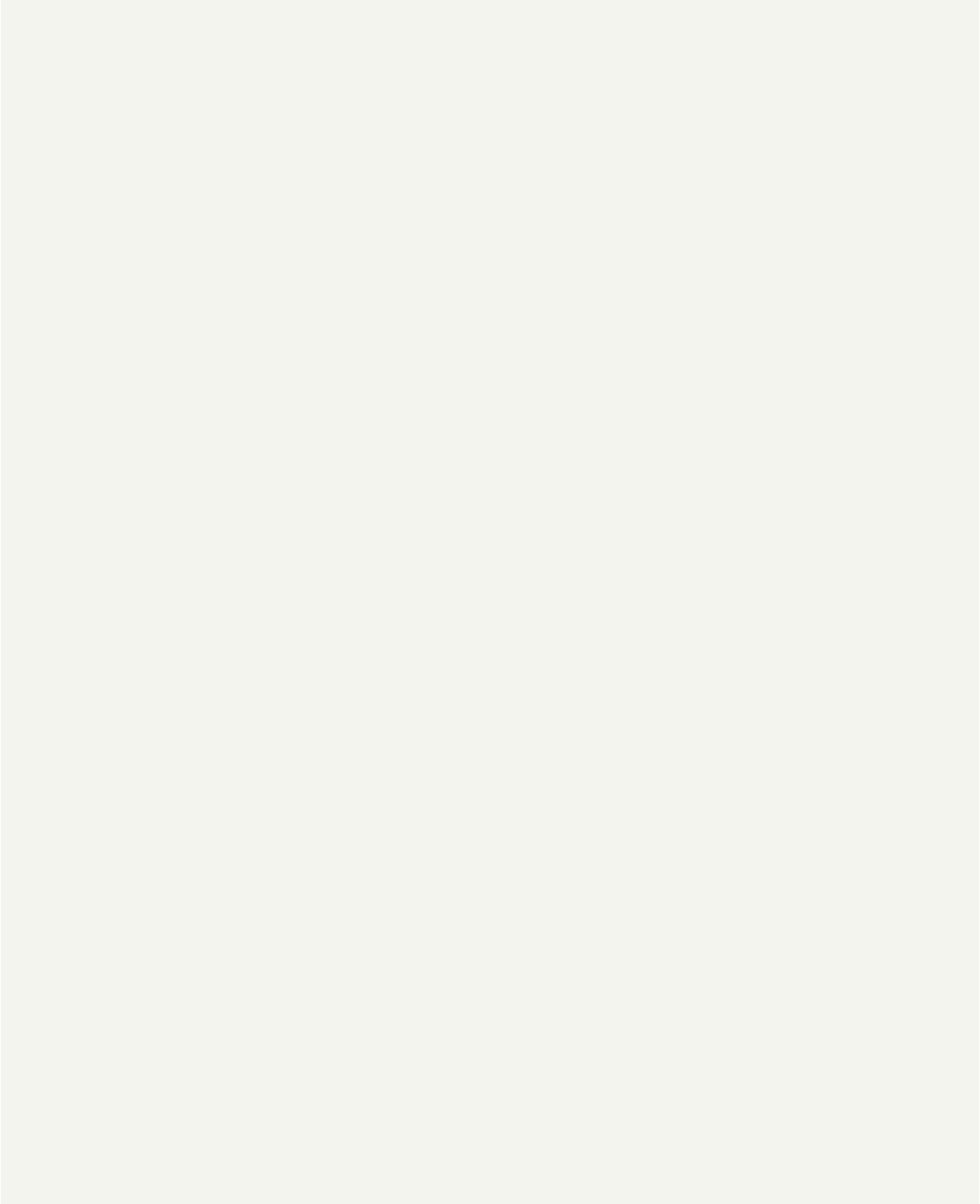
05

Seal, Shopify Admin,
Klaviyo, Supabase, App
Proxy

STACK

NEXT

Next.js 16 · Vercel ·
Supabase Postgres



ÍNDICE DEL DOCUMENTO

Seis secciones que cubren producto, arquitectura, superficies, integraciones y decisiones de diseño.

PORTAL LIT
DOCUMENTACIÓN · V1.0

01

RESUMEN EJECUTIVO

Qué es el portal, para quién es, y qué problema resuelve.

P. 03

02

STACK Y ARQUITECTURA

Next.js + Vercel + Supabase + Seal. Diagrama y razones detrás de cada pieza.

P. 04

03

AUTENTICACIÓN Y SESIONES

Dos vías de auth conviviendo: App Proxy y Customer Account API OAuth.

P. 06

04

SUPERFICIES DEL PORTAL

Mi LIT, Cuenta, Skip, Plan, Cancel, Login, Address, Payment. Pantalla por pantalla.

P. 08

05

INTEGRACIONES

Seal, Shopify Admin, Klaviyo, Supabase, App Proxy. Qué hace cada una.

P. 12

06

DECISIONES Y APRENDIZAJES

Los gotchas que aprendimos por las malas y por qué la solución es la que es.

P. 14



01 • RESUMEN EJECUTIVO

Qué es el portal, qué consigue el cliente, qué consigue LIT.

PORTAL LIT

SECCIÓN 01

El **Portal de Cliente de LIT** es la nueva área privada donde el suscriptor gestiona su caja: cambia el plan, salta envíos, modifica la dirección, añade extras, y, llegado el caso, cancela. Sustituye al portal genérico que venía por defecto con la herramienta de suscripciones (Seal), que vivía en un dominio distinto y rompía la identidad de marca.

EL PORQUÉ

El portal antiguo era una página técnica, fea, en otro dominio (`tracking.litsalt.com`). Tener uno propio significa que el cliente **jamás sale de litsalt.com**, ve la identidad LIT en todo momento, y nosotros controlamos cada flujo (cancelación, win-back, recompensas) en lugar de depender de lo que Seal ofrezca por defecto.

— LO QUE CONSIGUE EL CLIENTE

SELF-SERVICE

Total

Saltar caja, cambiar plan, dirección y pago en un click.

SIN SALIR DE MARCA

litsalt

Todo el flujo dentro de `/apps/portal/*`.

CANCELACIÓN

4 pasos

Con alternativas antes del fin: skip, plan, sabor.

VISIBILIDAD

100%

Próximas entregas, historial, sabor, dirección.

— LO QUE CONSIGUE LIT

- **Reducción de tickets a soporte** — la mayoría de "skip mi caja" / "cambiar plan" pasan a self-service.
- **Marca consistente** de extremo a extremo. Misma tipografía, mismo idioma, mismo trato.
- **Recuperación de cancelaciones integrada** en el propio flujo: el cliente ve alternativas antes de la decisión final.
- **Telemetría limpia**: cada acción se loguea en Klaviyo (skip, cancel reason, plan change) para alimentar flujos automáticos de retención.
- **Control sobre el roadmap**: Drops, Collection, Inner Circle, eventos en Madrid/Barcelona — fases siguientes ya con el chasis montado.

— ESTADO ACTUAL

El portal está **desplegado en producción** en `litsalt.com/apps/portal/*`. Los flujos críticos (login OAuth, ver suscripción, saltar, cambiar plan, cambiar dirección, cancelar) funcionan end-to-end. Hay una **auditoría de seguridad** ejecutada el 21 de mayo de 2026 con findings documentados aparte (`docs/SECURITY_AUDIT_2026-05-21.md`). Las fases siguientes — Collection (cartas), Drops (recompensas), Inner Circle (tier reward) — tienen el schema listo en Supabase y la UI con stubs.



02 · STACK Y ARQUITECTURA

Las piezas técnicas y por qué cada una está elegida.

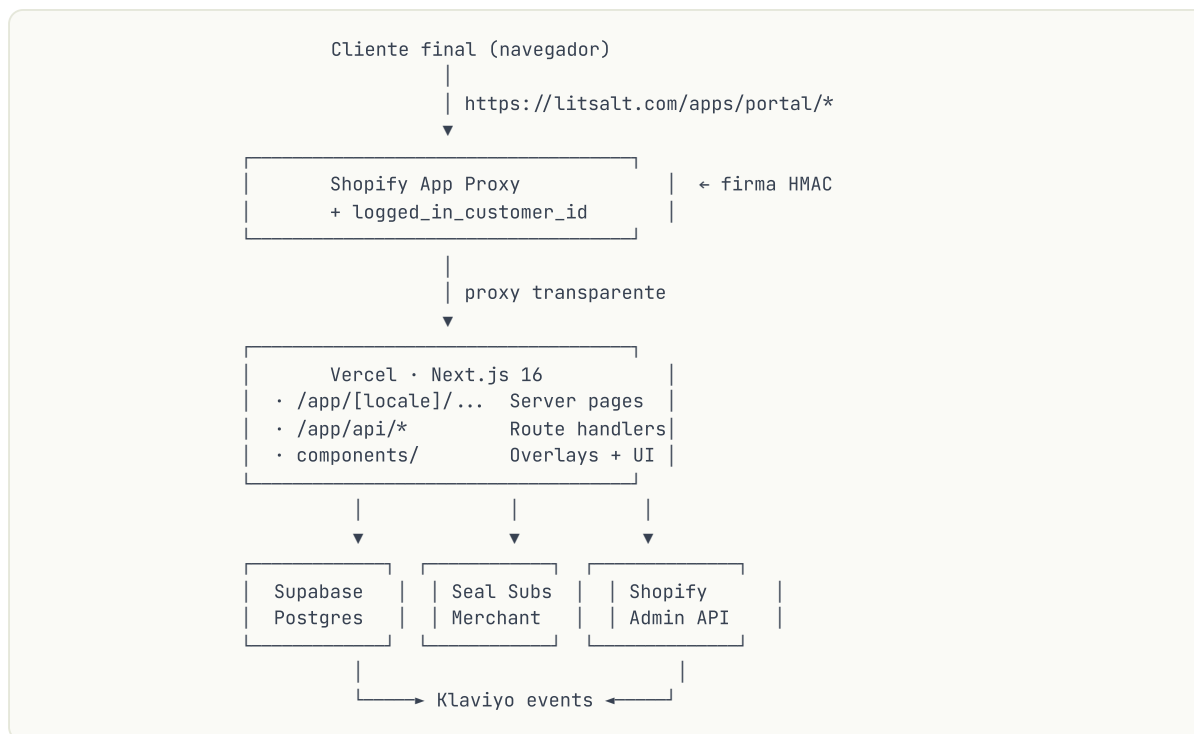
PORTAL LIT

SECCIÓN 02

— EL STACK EN UNA FRASE

Next.js 16 (App Router) en Vercel, servido al cliente vía **Shopify App Proxy**, con **Supabase Postgres** como base de datos, e integrado contra **Seal Subscriptions**, **Shopify Admin API** y **Klaviyo**.

— DIAGRAMA DE ALTO NIVEL



— LAS PIEZAS, UNA POR UNA

NEXT.JS 16

App Router. Server components para precargar datos sin exponer tokens al bundle del cliente. SSR para first paint rápido.

SHOPIFY APP PROXY

El cliente nunca sale de litsalt.com. Shopify firma cada request con HMAC y nos adjunta el customer_id.

VERCEL

Deploy continuo desde GitHub, escala automática, edge functions. Sin operar infra propia.

SUPABASE

Postgres gestionado + Auth + panel para ops review. Una sola DB para todo el portal.

SEAL SUBSCRIPTIONS

Motor de suscripciones existente. El portal habla con su Merchant API para leer y mutar.

CUSTOMER ACCOUNT API

OAuth moderno con PKCE de Shopify. Sustituye el login viejo. Soporta deep-links desde email.



02 · STACK Y ARQUITECTURA

Estructura del repositorio y convenciones.

PORTAL LIT
SECCIÓN 02 · CONT.

— ESTRUCTURA DEL REPOSITORIO

```
portal-cliente-web/lit-portal/
├─ src/app/
│  ├─ [locale]/           # páginas /es/* y /en/*
│  │  ├─ mi-lit/          # Hub: home del portal
│  │  ├─ cuenta/          # Account: datos del cliente
│  │  ├─ collection/      # Cartas (Fase 2)
│  │  ├─ the-world/       # Eventos/historias (Fase 2)
│  │  └─ auth/handoff/    # Transferencia de session_id desde callback
│  └─ api/                # Route handlers (mutations + reads)
│     ├─ auth/            # OAuth login/callback/logout
│     ├─ subscription/    # Plan, skip, cancel, extras, address
│     ├─ hub/dashboard    # Resumen agregado del Hub
│     └─ webhooks/        # Webhooks Seal + Shopify
├─ src/components/        # Overlays (PlanOverlay, SkipOverlay...), Hero, etc.
├─ src/lib/               # Clients: seal, shopify-admin, klaviyo, supabase
├─ database/
│  ├─ schema.sql          # Schema Postgres canónico
│  └─ migrations/         # Diff migrations aplicadas en prod
├─ scripts/               # Upload Klaviyo templates, debug, etc.
└─ docs/                  # Este documento + spec + auditoría
```

— CONVENCIONES

- **Idiomas:** bilingüe ES/EN vía componente `<T en="..." es="..." />`. URL incluye locale (`/es/mi-lit` , `/en/mi-lit`).
- **Mutations:** todas pasan por `withCustomer` middleware que verifica auth (App Proxy HMAC o Bearer/X-LIT-Session) antes de procesar.
- **Fast-path:** el cliente manda `sealSubscriptionId` en el body de mutations para que el backend evite el scan paginado de 33 páginas de Seal (~5s vs ~300ms).
- **Cutoff:** ventana de 24h antes del próximo envío durante la cual no se permiten cambios (skip, plan, dirección, extras). Constante única en `src/lib/cutoff.ts`.
- **Sin guiones largos** (em-dash) en NINGÚN copy. Decisión de marca. Usar coma o punto.

DECISIÓN DE MARCA

Toda la UI del portal usa las tipografías brand (**Barlow** y **Barlow Condensed**) y los colores LIT (**cream** `#F8F9F2` , **lit-grey** `#323743` , **bold-yellow** `#EBEE62`). No hay hojas de estilo "neutras" — todo es brand consistente.



03 · AUTENTICACIÓN Y SESIONES

Dos mecanismos de auth coexistiendo. Cualquiera identifica al cliente.

PORTAL LIT

SECCIÓN 03

El middleware `withCustomer` intenta dos vías de identificar al cliente. La primera funciona cuando el cliente entra desde el storefront de Shopify (típicamente desde un email). La segunda cuando el cliente entra directamente.

VÍA A · PASIVA

App Proxy

Shopify firma la URL con HMAC y adjunta

`logged_in_customer_id`. Transparente: si está logueado en la storefront, ya está dentro.

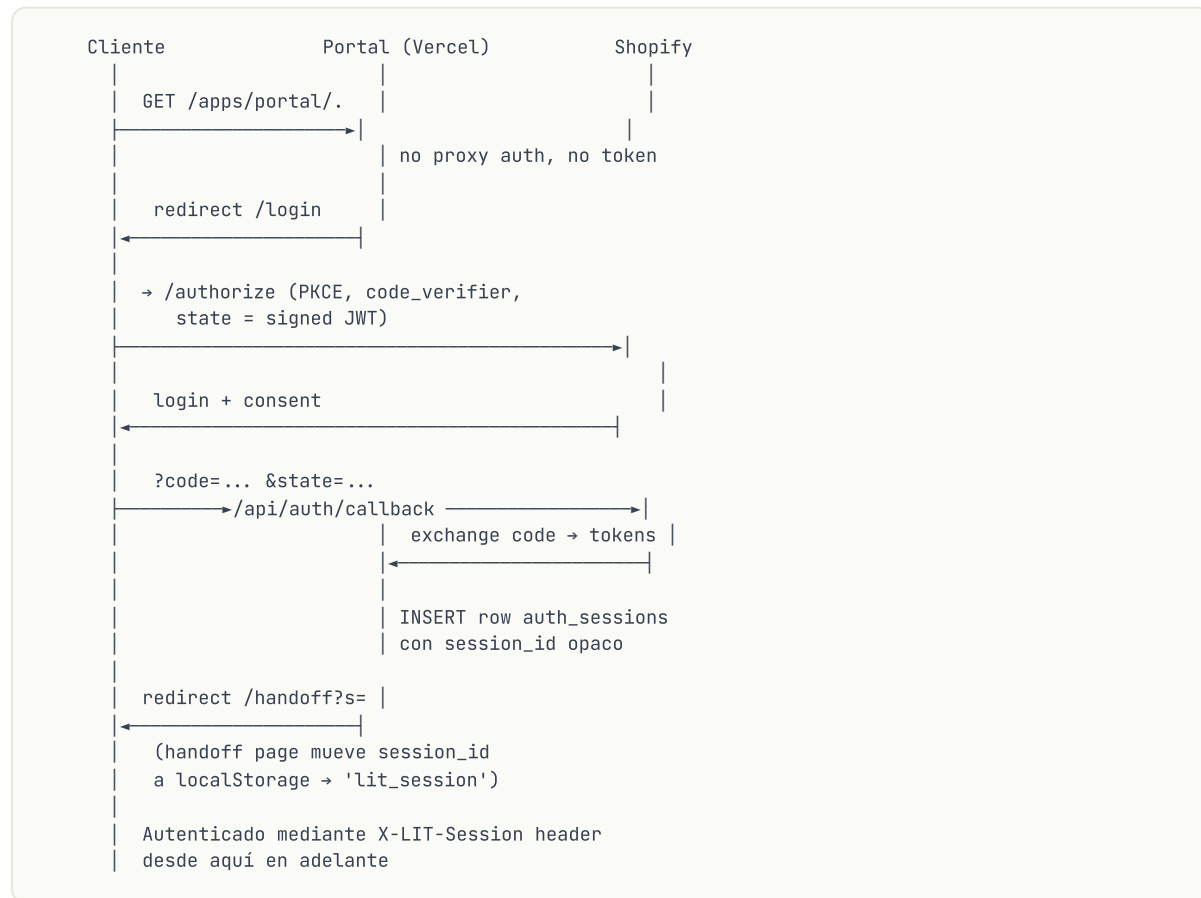
VÍA B · ACTIVA

OAuth (PKCE)

Customer Account API. Tras el callback creamos un

`session_id` opaco en Supabase. El FE lo manda en `X-LIT-Session`.

DIAGRAMA DEL FLUJO OAUTH (VÍA B)



POR QUÉ `X-LIT-SESSION` Y NO `AUTHORIZATION: BEARER`

GOTCHA SHOPIFY INTERCEPTA AUTHORIZATION EN MUTATIONS

Shopify App Proxy **intercepta el header** `Authorization` en peticiones POST/PATCH/DELETE y NO las reenvía al upstream. En su lugar devuelve un 500 con el HTML del tema (la home de la tienda), lo cual hace que el FE crea que el

endpoint está roto. Headers custom `X-*` pasan limpios. Migramos a `X-LIT-Session` y todas las mutations volvieron a funcionar al instante.

03 · AUTENTICACIÓN Y SESIONES

Logout, sesiones en Supabase, y guardas server-side.

PORTAL LIT
SECCIÓN 03 · CONT.

TABLA AUTH_SESSIONS (SUPABASE)

COLUMNA	TIPO	PROPÓSITO
session_id	text PK	Token opaco (32 bytes random hex) que el FE guarda en localStorage.
customer_id	text	Shopify customer_id (numeric, p.ej. 27453541548381).
email	text	Email del cliente (cacheado del id_token).
id_token	text	Token OIDC original, usado en el logout para invalidar la sesión storefront.
created_at	timestampz	Cuándo se emitió la sesión.
expires_at	timestampz	TTL de 30 días. <code>withCustomer</code> rechaza si está caducada.
last_used_at	timestampz	Best-effort refresh por cada request autenticada.

LOGOUT

El endpoint `/api/auth/logout`:

- Borra la fila de `auth_sessions` en Supabase.
- Construye la URL de logout OIDC de Shopify con `id_token_hint` + `post_logout_redirect_uri`.
- El FE limpia su `localStorage` (`clearSessionToken`) y hace `window.location.replace` a esa URL.
- Shopify invalida la sesión storefront y redirige al cliente al destino configurado.

CASO ESPECIAL

Tras una **cancelación**, el botón "Volver a LIT" del done state ejecuta el flujo de logout completo y manda al cliente a `litsalt.com` (homepage). No quiero que el cliente vuelva al portal y vea estado *stale* (sub "todavía activa" en la UI local mientras Seal ya la canceló). Salir limpio.

SUB-GUARD: PROTECCIÓN CONTRA IDOR

El helper `assertSubscriptionBelongsToCustomer` (`src/lib/sub-guard.ts`) se ejecuta antes de CADA mutation sobre una sub. Compara el `sub.email` de Seal contra el email del Shopify customer autenticado. Si no matchean → 403 inmediato y log de auditoría con ambos emails. Se añadió tras un *near-incident* en el que Seal devolvió la sub de otro cliente en una pagination edge case.



04 · SUPERFICIES DEL PORTAL

Las 8 pantallas del cliente, una por una. Hub y Cuenta.

PORTAL LIT

SECCIÓN 04 · 1/3

Cada superficie tiene una URL clara y una responsabilidad. Todas comparten header móvil fijo, mismo sistema de tipografías (Barlow Condensed títulos, Barlow body) y colores brand (cream / lit-grey / bold-yellow).

4.1 · MI LIT (HUB)

HOME

RUTA PRIMARIA

/apps/portal/[es|en]/mi-lit

Pantalla principal del cliente. Estructura:

- **Hero "Next Box"**: fecha grande del próximo envío, sabor, plan. 4 variantes según estado — `default`, `skipped` (con banner amarillo + undo), `locked` (dentro de 24h), `new` (primera caja).
- **Quick Actions**: 4 botones — Cambiar plan, Saltar próxima, Cambiar sabor (Fase 2), Extras (Fase 2). Cada uno abre su overlay.
- **Calendar de próximas entregas**: las próximas 5-6 fechas que Seal tiene pre-agendadas.
- **Mini Collection grid**: preview de la colección de cartas (Fase 2).
- **Order history**: últimos 10 pedidos completados con tracking si está disponible.

4.2 · CUENTA

ACCOUNT

SETTINGS

/apps/portal/[es|en]/cuenta

Misma información que Mi LIT en formato ficha: nombre, email, teléfono, suscripción (cajas, frecuencia, sabor, próxima fecha), dirección, método de pago, historial completo de pedidos. Incluye acceso rápido al plan overlay y, al final, la **Danger Zone** con "Cerrar sesión" y "Cancelar suscripción".

- Los **Quick Actions** funcionan exactamente igual que en Mi LIT (mismas reglas de disabled, mismo banner "Saltaste la entrega anterior — Deshacer" si hay skip activo).
- El banner del skip se sincroniza vía `localStorage` ('lit:just-skipped'): si saltas desde Mi LIT y navegas a Cuenta lo ves; viceversa también.



04 · SUPERFICIES DEL PORTAL

Skip, Plan, Address, Payment, Extras — las mutations.

PORTAL LIT

SECCIÓN 04 · 2/3

4.3 · SKIP OVERLAY

OVERLAY

MUTATION

POST /api/subscription/skip

Bottom-sheet en móvil, modal centrado en desktop. Muestra "¿Te vas de viaje?", la fecha de envío actual, y un botón "Confirmar saltar". Al confirmar:

- POST con `sealSubscriptionId` en el body (fast-path).
- Backend valida ownership en Supabase, hace GET dirigido a Seal por ID (sin paginar), comprueba cutoff de 24h, llama `skipBillingAttempt`.
- FE recibe la nueva fecha, marca `justSkipped` en localStorage con expiry hasta esa fecha, dispara evento Klaviyo.

4.4 · PLAN OVERLAY

OVERLAY

MUTATION

PATCH /api/subscription/plan

Selector de cajas (1 a 6) + selector de frecuencia (15d hasta 6 meses). Precio dinámico desde Shopify con `compareAtPrice` tachado si aplica descuento. Si hay skip activo, banner amarillo: "Al cambiar el plan, tu salto de caja se deshará". Al guardar:

- Si cambia **frecuencia**: `seal.editSubscription(delivery_interval)` primero.
- Si cambia **nº de cajas**: `addItem`s con nueva variant, luego `removeItems` de la antigua. Seal no permite swap directo.
- Pausas de 500ms entre mutations porque Seal regenera `billing_attempts` en background.
- Verify post-mutation con AbortController de 4s. Si timed out → respuesta sintética + FE arranca re-poll silencioso de 60s.

4.5 · ADDRESS OVERLAY

OVERLAY

MUTATION

PATCH /api/subscription/address

Formulario para cambiar dirección de envío. PATCH llama a `seal.updateShippingAddress`. Aplica cutoff de 24h.

4.6 · PAYMENT METHOD

EMAIL-ONLY

Shopify `customerPaymentMethodSendUpdateEmail`

Decisión locked 2026-05-19: **100% email-only**. El botón "Cambiar método de pago" dispara Shopify Admin API, que envía un enlace seguro al email del cliente. El portal muestra: "Te hemos enviado un enlace a tu correo, vuelve aquí al terminar". Sin Stripe, sin pasarela nueva.



04 · SUPERFICIES DEL PORTAL

El takeover de cancelación: el flujo más cuidado del portal.

PORTAL LIT
SECCIÓN 04 · 3/3

4.7 · CANCEL TAKEOVER

FULL-SCREEN WIN-BACK MULTI-STEP

/api/subscription/cancel (steps 1 → 4)

No es un overlay, es un **takeover oscuro a pantalla completa** que ocupa toda la viewport. 4 pasos:

STEP 01

Stats

"Has recibido N cajas y llevas M meses en LIT". Reconocer el viaje del cliente antes de proponer salirse.

STEP 02

Alternativas

Botones para Saltar, Cambiar plan o Sabores. Pivotan al overlay correspondiente SIN cerrar el takeover.

STEP 03

Motivo

Selector de razón (caro / mucho producto / no lo uso / pausa / otro) + free text opcional.

STEP 04

Confirmación

Copy condicional según cutoff. Botón final "Cancelar suscripción".

— QUÉ PASA AL CONFIRMAR

1. Marca la cancellation row en Supabase como `committing` (estado intermedio para detectar cancels colgados).
2. Llama `seal.cancelSubscription` (es **inmediato** — Seal pone `cancelled_on` al momento).
3. Promueve la row a `confirmed` con timestamp.
4. Si es **second-cancel**: reset inmediato del balance Drops. Si es first: hold de 90 días.
5. Dispara evento Klaviyo `subscription_cancelled` con razón, freeText, cancelCount, fecha último envío, drops hold.
6. Muestra el done state: *"Muchas gracias por haber confiado en LIT. Ojalá poder tenerte de vuelta pronto."*

SALIDA LIMPIA

El botón "Volver a LIT" del done state hace **logout completo** (borra session en Supabase, limpia localStorage) y redirige a `litsalt.com`. El cliente NO vuelve al portal con estado stale. Si más tarde quiere volver a entrar, el Hub le mostrará el ReactivateCard.

4.8 · LOGIN / EMPTYSTATE

FALLBACK

/apps/portal/[es|en]/mi-lit (sin auth)

Sin sesión → **LoginScreen** con CTA a OAuth. Autenticado pero sin suscripción activa (one-shot buyer o cancelado) → **EmptyState** con *"Gracias por probar LIT"* + CTA a la página de suscripción con descuento "desde el 25%".



05 • INTEGRACIONES

Seal Subscriptions y Shopify Admin API.

PORTAL LIT

SECCIÓN 05 • 1/2

5.1 • SEAL SUBSCRIPTIONS (MERCHANT API)

El motor de suscripciones de LIT. El portal lee y muta el estado de la suscripción únicamente a través de su Merchant API.

ACCIÓN	ENDPOINT	NOTAS
Listar subs por email	GET /subscriptions?page=N	Sin filtro server-side. Hay que paginar hasta 33 páginas para cuentas grandes.
Sub por ID	GET /subscription?id=X	Endpoint SINGULAR que sí honra el filtro. Evita la paginación. Descubierto 2026-05-21.
Cambiar intervalo	PUT /subscription action=edit	Solo <code>delivery_interval</code> . NO mandar <code>billing_interval</code> – Seal silenciosamente no aplica el cambio.
Cambiar variant	add_items + remove_items	No hay endpoint de swap. Añadir nuevo + quitar viejo, en ese orden.
Saltar attempt	PUT /subscription-billing-attempt action=skip	Puede devolver 200 con <code>success: false</code> . Hay que chequear el body, no solo el status.
Cancelar / reactivar	PUT /subscription action=cancel reactivate	Cancel es INMEDIATO . Seal pone <code>cancelled_on</code> al momento.

5.2 • SHOPIFY ADMIN API (GRAPHQL)

Usada para datos del cliente y operaciones que Seal no expone.

- **Lookup de email** por `customer_id` – App Proxy nos da el ID, pero Seal indexa por email.
- **Actualizar customer** – nombre, teléfono, language preference.
- **Catalog de extras** – productos con tag `add-to-box` añadibles a la siguiente caja.
- **Send payment method update email** – `customerPaymentMethodSendUpdateEmail` para el flujo de cambio de pago.
- **Order history** – últimos pedidos del cliente para el historial.



05 • INTEGRACIONES

Klaviyo (telemetría + emails) y Supabase (datos del portal).

PORTAL LIT
SECCIÓN 05 • 2/2

5.3 • KLAVIYO

Cada acción importante del portal dispara un evento custom en Klaviyo, que alimenta flujos automáticos de marketing y win-back. Eventos actuales:

EVENTO	CUÁNDO	PROPERTIES
subscription_skip	Tras skip exitoso	newNextShipDate, sealSubscriptionId
subscription_cancelled	Tras cancel step 4	primaryReason, freeText, cancelCount, lastShipDate, dropsHeldUntil
welcome_to_portal	Primera sesión OAuth	customerId, email, locale

Las plantillas de email viven en Klaviyo y se sincronizan con el repo vía `scripts/klaviyo-upload-templates.mjs`. Hay dos plantillas welcome (ES `VXC2NN`, EN `SCE2Q2`) con CTA al portal y footer "JOIN THE LIT MOVEMENT" alineado a la marca.

5.4 • SUPABASE (POSTGRES)

Tablas principales del portal:

TABLA	PROPÓSITO
auth_sessions	Mapping session_id ↔ customer_id, expiry, id_token para logout OIDC.
subscriptions	Cache del mapping customer_id ↔ seal_subscription_id. Evita el scan de 33 páginas en cada request.
cancellations	Multi-step cancel: pending → committing → confirmed. Detecta cancels colgados.
customer_preferences	cancel_count, language_pref, last_cancelled_at. Lógica de "second-cancel = Drops reset inmediato".
drops_balances	Balance de Drops por customer (Fase 2 activa cuando los triggers estén listos).
drops_events	Log de cada cambio de balance: box shipped, referral converted, monthly streak, etc.
webhook_log	Idempotency log de webhooks Shopify y Seal. Evita procesar el mismo evento dos veces.

5.5 • SHOPIFY APP PROXY

Configurado en `shopify.app.toml`:

```
[app_proxy]
url      = "https://lit-portal-drab.vercel.app"
subpath  = "portal"
prefix   = "apps"
```

Cualquier request a `litsalt.com/apps/portal/*` se reenvía con HMAC + `logged_in_customer_id` + `shop` + `timestamp`. El middleware `verifyAppProxyRequest` valida la firma antes de procesar.



06 · DECISIONES Y APRENDIZAJES

Los gotchas que aprendimos por las malas. Para que no se repitan.

PORTAL LIT

SECCIÓN 06 · 1/2

GOTCHA 01 SHOPIFY APP PROXY SUSTITUYE 5XX CON HTML DEL STOREFRONT

Cuando el upstream (Vercel) devuelve un HTTP 5xx, App Proxy **NO reenvía esa respuesta al cliente**. En su lugar, renderiza el HTML del tema de la tienda con la URL solicitada como canonical. El cliente recibe ~50KB de markup del storefront con status 500, pero `content-type: text/html` y headers de Shopify(`powered-by: Shopify`, `server-timing: theme;desc=...`).

Diagnóstico: bypass del proxy y hablar directo con Vercel(`curl https://lit-portal-drab.vercel.app/...`) para ver el error JSON real. Nos costó horas descubrir esto.

GOTCHA 02 SEAL REGENERA BILLING_ATTEMPTS EN CADA PLAN CHANGE

Cualquier `editSubscription` en Seal (cajas, frecuencia, dirección, items) hace que regenere la lista de `billing_attempts` desde cero. **Cualquier skip aplicado previamente se borra silenciosamente.**

Mitigación: el FE limpia el flag `justSkipped` de localStorage cuando hay plan change exitoso, y el PlanOverlay avisa con banner amarillo si hay skip activo antes de cambiar plan.

GOTCHA 03 EVENTUAL CONSISTENCY EN SEAL

Tras una mutation, un GET inmediato puede devolver datos stale (un attempt aparece como pendiente cuando ya está marcado skipped).

Soluciones: (1) el skip endpoint **computa el nuevo nextShipDate localmente** en vez de re-fetch; (2) el plan endpoint usa AbortController con timeout de 4s; (3) el FE arranca re-poll silencioso de 60s tras un plan change.

GOTCHA 04 CONSTRAINT CHECK SIN MIGRATION APLICADA EN PROD

El código del cancel step 4 escribía `status='committing'` antes de llamar a Seal, pero la constraint CHECK del schema solo permitía `('pending','confirmed')`. Durante semanas **nadie podía cancelar** — el portal devolvía 500 que Shopify convertía en HTML del storefront (Gotcha 01 enmascarando esto).

Fix: `ALTER CONSTRAINT` aplicado el 2026-05-21. Migration en `database/migrations/`. Schema canónico sincronizado.

GOTCHA 05 CANCEL ES INMEDIATO EN SEAL (NO DESPUÉS DEL ÚLTIMO ENVÍO)

El comportamiento por defecto de Seal cancel pone `cancelled_on` al instante, no tras el último envío. El portal refleja esto en el copy de confirmación según el cutoff: *"Cancelación inmediata"*, excepto si el próximo envío ya está procesando dentro de 24h, en cuyo caso ese último sí sale.



06 · DECISIONES Y APRENDIZAJES

Las reglas duras de marca y producto.

PORTAL LIT
SECCIÓN 06 · 2/2

REGLA 06 EL CLIENTE JAMÁS VE TRACKING.LITSALT.COM

Regla dura: el cliente final no puede ver, en ningún momento, el portal de tracking antiguo. Todo el flujo de su cuenta vive dentro de `litsalt.com/apps/portal/*`. Cambio de método de pago = email. Cancel = takeover. Error = inline. El portal viejo "dejará de existir" para el cliente final.

REGLA 07 SIN EM-DASHES (—) EN NINGÚN COPY

Hard rule de marca: ningún copy de LIT (portal + emails + ads) puede llevar un em-dash. Usar coma o punto. Preferencia consistente del founder. Lint cuando se genera contenido nuevo.

REGLA 08 CUTOFF BAJADO DE 72H A 24H

Decidido el 2026-05-21: el operador interno necesita al menos 24h para procesar cambios antes del envío. Antes eran 72h, que bloqueaba demasiados cambios que sí se podían atender. Se aplica a skip, plan, address, extras. Constante única en `src/lib/cutoff.ts`.

REGLA 09 CAMBIO DE MÉTODO DE PAGO = EMAIL-ONLY

Decisión locked 2026-05-19: el cambio de método de pago es 100% email-only via Shopify `customerPaymentMethodSendUpdateEmail`. Sin Stripe, sin pasarela nueva. Shopify ya tiene el método de pago y la pasarela — el portal solo dispara el envío del enlace seguro.

REGLA 10 AUTH SEGURA POR DEFECTO: X-LIT-SESSION NO AUTHORIZATION

Tras el descubrimiento del 2026-05-21 (Shopify App Proxy bloquea Authorization en mutations), el portal envía el session token en header custom `X-LIT-Session`. El backend acepta ambos para back-compat, pero el FE solo usa el custom. Esto resolvió de raíz una clase entera de bugs ("la mutation parece que funciona pero nunca llega al servidor").

CIERRE

El portal está vivo, los flujos críticos funcionan end-to-end, y el cliente puede gestionar su suscripción sin ver el portal antiguo. La base es sólida. Las decisiones documentadas en estas dos páginas son las que evitarán que el equipo repita los mismos errores cuando llegue Fase 2 (Collection, Drops, Inner Circle) o cuando se monte un nuevo portal para otra marca con la misma arquitectura.



L I T ■

FIN DEL DOCUMENTO

GRACIAS POR LEER.

Para detalles de seguridad ver `docs/SECURITY_AUDIT_2026-05-21.md` en el repo.

PORTAL LIT · DOC V1.0 · 21 · 05 · 2026

